

HelixSpecifier

HelixSpecifier

कार्य के अनुरूप अपनी औपचारिकता को स्वयं अनुकूलित करने वाला स्पेक-चालित विकास।

सारांश

HelixSpecifier एक Go इंजन है जो तीन विकास पद्धतियों — SpecKit का स्पेक-चालित कार्यप्रवाह, Superpowers की TDD अनुशासन, और GSD का मील का पत्थर जीवनचक्र — को एक अनुकूलनीय प्रवाह में समाहित करता है। यह प्रत्येक कार्य को प्रयास के आधार पर वर्गीकृत करता है और उसके अनुसार प्रक्रिया की मात्रा को बढ़ाता या घटाता है।

संक्षिप्त विवरण

HelixSpecifier AI एजेंटों के लिए एक स्पेक-चालित विकास संलयन इंजन है। यह SpecKit, Superpowers और GSD को जोड़ता है, कार्य को प्रयास स्तर के आधार पर वर्गीकृत करता है, बहस-समर्थित स्पेसिफिकेशन चरण चलाता है, न्यूनतम परीक्षण-कोड अनुपात लागू करता है, और हर पूर्ण प्रवाह से सीखता है।

विस्तृत विवरण

HelixSpecifier एक स्पेक-चालित विकास (SDD) संलयन इंजन है जिसे Go (मॉड्यूल `digital.vasic.helixspecifier`) में लिखा गया है, और यह HelixAgent AI समूह का एक घटक है। यह तीन विकास पद्धतियों को — जो सामान्यतः तीन अलग-अलग उपकरणों और तीन अलग-अलग मानसिकताओं में काम करती हैं — एक अनुकूलनीय कार्यप्रवाह में समाहित करता है: SpecKit की सात-चरणीय SDD प्रक्रिया (Constitution, स्पेसिफाई, क्लैरिफाई, प्लान, टास्कस, एनालाइज़, इम्प्लीमेंट), Superpowers का परीक्षण-चालित अनुशासन जिसमें समानांतर उप-एजेंट निष्पादन शामिल है, और GSD का मील का पत्थर जीवनचक्र प्रबंधन। प्रत्येक स्तंभ वही करता है जिसमें वह निपुण है; इंजन ही उन्हें एक एकीकृत प्रवाह के रूप में चलाता है, न कि हाथ से सिले हुए पाइपलाइन के रूप में।

इसकी केंद्रीय अवधारणा है अनुकूलनीय औपचारिकता: इंजन आने वाले कार्य को प्रयास स्तर के आधार पर वर्गीकृत करता है और प्रक्रिया की मात्रा को उसके अनुरूप समायोजित करता है, ताकि एक पंक्ति का सुधार उसी भारी-भरकम प्रक्रिया से न गुजरे जिससे एक बड़ा फीचर

गुजरता है — और न ही एक बड़ा फीचर टाइपो की तरह लापरवाही से पारित हो जाए। इस मूल संरचना पर दस शक्तिशाली विशेषताएँ आधारित हैं: सीमित-समानता वाले समानांतर कार्य निष्पादन, मशीन-पठनीय “Constitution ऐज़ कोड” जो स्वचालित रूप से अनिवार्य नियम लागू करता है, “नाइक्विस्ट TDD” जो परीक्षण-कार्यान्वयन अनुपात को ट्रैक और लागू करता है, स्पेसिफिकेशन परिष्करण के लिए बहु-चरणीय बहु-एजेंट बहस, अनुकूलनीय कौशल-प्रवीणता सीखना, पुराने कोड का ब्राउनफील्ड विश्लेषण, ऐतिहासिक पैटर्न से पूर्वानुमानित स्पेसिफिकेशन, परियोजना-आइ ज्ञान हस्तांतरण, रनटाइम में औपचारिकता का पुनः-समायोजन जो उड़ान के बीच में पुनः-ट्यूनिंग करता है, और अर्थपूर्ण खोज के साथ स्थायी स्पेक स्मृति।

यह Go मॉड्यूल के रूप में उपयोग किया जाता है — go get या स्थानीय प्रतिस्थापन निर्देश के माध्यम से — एक जानबूझकर छोटे इंजन API के पीछे: तीन स्तंभों, एक औपचारिकता स्केलर और स्पेक स्मृति को पंजीकृत करें, कार्य के प्रयास को वर्गीकृत करें, फिर पूर्ण प्रवाह निष्पादित करें और गुणवत्ता-स्कोरित परिणाम प्राप्त करें। सतह सरल है; इसके पीछे का ऑर्केस्ट्रेशन नहीं। Helix परिवार की तरह, इसे एक एंटी-ब्लफ़ सत्यापन व्यवस्था के तहत विकसित किया गया है, जिसमें इन-प्रोसेस चैलेंज रनर वास्तविक कोड का उपयोग करता है न कि मॉक का।

हमने इसे क्यों बनाया

स्पेक-चालित विकास, कठोर TDD, और मील का पत्थर प्रबंधन आमतौर पर तीन अलग-अलग पद्धतियाँ हैं जिनके लिए तीन अलग-अलग उपकरणों की आवश्यकता होती है। HelixSpecifier इसलिए बनाया गया ताकि एक AI एजेंट (HelixAgent) इन तीनों को एक सुसंगत, स्वतः-समायोजित कार्यप्रवाह के रूप में चला सके, न कि उन्हें हाथ से जोड़कर।

सामग्री

यह क्यों एक गेम-चेंजर है

यह प्रक्रिया को काम के अनुपात में स्वचालित रूप से ढाल देता है। आमतौर पर टीमों दो बुरी चरम स्थितियों में से किसी एक में फँस जाती हैं: हर चीज़ पर भारी औपचारिकता (सुरक्षित लेकिन धीमी, और चुपचाप नाराज़गी भरी) या फिर किसी पर भी औपचारिकता नहीं (तेज़ जब तक कि अचानक न हो जाए)। HelixSpecifier इस समझौते को समाप्त कर देता है — हर कार्य की वर्गीकृत मेहनत के अनुसार औपचारिकता का आकार तय करता है और काम के प्रकट होने पर उसे रनटाइम पर पुनः समायोजित करता है। जो क्षमता पहले व्यावहारिक नहीं थी, वह है हर कार्य के लिए स्वतः समायोजित प्रक्रिया — और इसके अलावा, एकल एजेंट के प्रथम अनुमान के बजाय बहु-चरणीय, बहु-एजेंट बहस और स्थिति स्कोरिंग द्वारा समर्थित विनिर्देश निर्णय।

क्या है नवीन

- अनुकूली औपचारिकता — प्रक्रिया स्तर वास्तविक समय गुणवत्ता मापदंडों द्वारा संचालित और रनटाइम पर समायोजित, पहले से तय नहीं।
- नाइक्विस्ट टीडीडी — टेस्ट-टू-इम्प्लीमेंटेशन अनुपात गेट (न्यूनतम 2x), नाइक्विस्ट सैंपलिंग प्रमेय के तर्क को अपनाते हुए: व्यवहार को सटीकता से पकड़ने के लिए उसे

उसकी दर से कहीं अधिक दर पर सैंपल करना ज़रूरी है, इसलिए टेस्ट उस कोड से कहीं अधिक मापने चाहिए जिसे वे कवर करते हैं।

- **बहस वास्तुकला** — बहु-चरणीय, बहु-एजेंट विनिर्देश परिष्करण, जहाँ स्थितियाँ प्रस्तावित, स्कोर की जाती हैं और एक-दूसरे के विरुद्ध अभिसरित होती हैं, एकल राय को प्रतिद्वंद्वी दृष्टिकोण से बदल देती हैं।
- **पूर्वानुमानित विनिर्देश और परियोजना-आंतरिक हस्तांतरण** — इंजन संचित प्रवाहों का विश्लेषण करके विनिर्देशों का पूर्वानुमान लगाता है और एक परियोजना से प्राप्त कठिन ज्ञान को अगली परियोजना में स्थानांतरित करता है।
- **Constitution कोड के रूप में** — अनिवार्य परियोजना नियम मशीन-पठनीय बनाए जाते हैं और इंजन द्वारा लागू किए जाते हैं, समीक्षक की सतर्कता पर निर्भर नहीं।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **तीन पद्धतियों को एक-दूसरे से टकराए बिना जोड़ना** — SpecKit, Superpowers और GSD प्रत्येक यह मानते हैं कि कार्यप्रवाह पर उनका अधिकार है। इसका समाधान एक फ्यूजन इंजन से किया गया, जो प्रत्येक स्तंभ को एक साझा इंटरफ़ेस के पीछे पंजीकृत करता है और उन्हें एक ही प्रवाह जीवनचक्र के माध्यम से संचालित करता है, ताकि वे तीन टकराने वाली प्रक्रियाओं के बजाय एक ही प्रक्रिया में समाहित हो जाएँ।
- **यह तय करना कि किसी कार्य के लिए वास्तव में कितनी प्रक्रिया चाहिए** — अधिक अनुमान लगाने पर सब कुछ धीमा हो जाता है; कम अनुमान पर जोखिम भरा काम बिना जाँच के निकल जाता है। इसका समाधान एक प्रयास वर्गीकरणकर्ता से किया गया, जो काम का आकार तय करता है और एक औपचारिकता स्केलर को फ़ीड करता है, जो निष्पादन के दौरान प्रक्रिया स्तर को गतिशील रूप से समायोजित करता है।
- **विनिर्देश की गुणवत्ता को उच्च बनाए रखना बिना हर निर्णय पर मानव नियंत्रक के** — इसका समाधान एकल-चरण विनिर्देशों को बहस-समर्थित परिष्करण से बदलकर किया गया, जहाँ एजेंट कई चरणों में प्रतिस्पर्धी स्थितियों को स्कोर करते हैं, और नाइक्विस्ट टीडीडी अनुपातों को लागू करके यह सुनिश्चित किया जाता है कि कार्यान्वयन अपने टेस्ट से आगे न बढ़ सके।

तकनीकी ढाँचा

- **Go** — चुना गया ताकि इंजन एकल आयात योग्य बाइनरी के रूप में आए, जिसमें कोई रनटाइम बोझ न हो; इसकी समवर्ती मॉडल ही सीमित-समानांतर कार्य वितरण और बहु-एजेंट बहस चरणों को थ्रेडिंग की जटिलता के बजाय व्यावहारिक बनाता है।
- **logrus** — इंजन और तीनों स्तंभों में संरचित लॉगिंग, ताकि प्रवाह के निर्णय (वर्गीकरण, औपचारिकता परिवर्तन, बहस परिणाम) बाद में स्पष्ट रूप से पढ़े जा सकें।
- **SpecKit स्तंभ** — सात-चरणीय विनिर्देश-संचालित विकास प्रक्रिया (Constitution → विनिर्देशित करें → स्पष्ट करें → योजना बनाएँ → कार्य → विश्लेषण करें → कार्यान्वित करें), जो विनिर्देश से कोड बनने की अनुशासित रीढ़ प्रदान करता है।
- **Superpowers स्तंभ** — टीडीडी अनुशासन के साथ समानांतर उप-एजेंट निष्पादन, जो टेस्ट-फ़स्ट कठोरता और वह विस्तार प्रदान करता है जो कार्यान्वयन को ईमानदार और तेज़ रखता है।
- **GSD स्तंभ** — मील के पत्थर और जीवनचक्र प्रबंधन, जो प्रवाह को “पूर्ण” की भावना और चरणों के माध्यम से प्रगति प्रदान करता है।

- **विनिर्देश स्मृति भंडार** — पिछले विनिर्देशों का एक स्थायी, अर्थपूर्ण रूप से खोजने योग्य सूचकांक, वह आधार जो पूर्वानुमानित विनिर्देश और परियोजना-आंतरिक हस्तांतरण को हर बार शून्य से शुरू करने के बजाय संभव बनाता है।

सामग्री

स्थिति एवं ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। इसे HelixAgent के Go मॉड्यूल घटक के रूप में उपयोग किया गया।
- **लाइसेंस:** निर्धारित किया जाना शेष। GitHub API के माध्यम से कोई लाइसेंस नहीं पाया गया — अपुष्ट / घोषित नहीं।
- प्रदर्शित नाम “HelixSpecifier” रिपॉजिटरी specifier से संबद्ध है।

प्राथमिकता स्तर: Helix-प्राथमिक।